华东师范大学软件学院实验报告

实验课程：计算机网络	年级：2024 级	实验成绩：
实验名称: IPv4	姓名：刘小凡	实验日期：2026-10-13
实验编号：Lab3	学号：10244602411	实验时间：2 学时

一、实验目的

二、实验内容与实验步骤

三、实验环境

四、实验过程与分析

本次实验围绕 IPv4 协议展开，目的是通过抓取并逐字段检查真实的 IPv4 数据报，掌握 IP 首部的结构与各字段含义，并学会用抓包证据回答关于寻址、长度、标识、生存时间与分片的问题。具体目标如下：

1. 掌握在 Wireshark 中选择联网接口、设置 tcp port 80 捕获过滤器、开启网络名称解析并关闭混杂模式的完整抓包流程，能够保存可复核的 pcapng 迹线；

2. 能够把 IPv4 首部各字段与被高亮的原始字节逐一对应，理解哪些字段共享同一个字节，并据此还原首部长度字段 IHL 的编码方式；

3. 通过实测数据总结 Total Length、Identification、TTL 以及分片标志的变化规律，并区分“未分片”的判据与 IP 校验和卸载等容易误判的现象；

4. 使用 traceroute 类探测发现从本机到远端服务器的路由器级路径，理解星号跳的含义，并能借助地址登记信息为每一跳标注归属组织；

5. 独立完成一次 IPv4 首部校验和的 1 的补码求和验算，把 10 个 16 位字与对应字段一一列出，验证结果等于 0xffff。

实验按手册的五个 Step 推进：先用 curl 抓取同一远端服务器的网页并用 tcp port 80 过滤器捕获，再做 traceroute 探测路径；随后在 Wireshark 中展开 IPv4 首部逐字段核对；接着绘制 IP 包结构图并回答六道问题；之后依据路径探测结果绘制网络路径图；最后选取一个接收方向、非零校验和、20 字节首部的包完整验算 IP 首部校验和。

本次实测使用 curl 代替手册中的 wget，两者都是命令行取回工具，作用等价。为避免域名解析或代理影响结果，抓包时用 --resolve 把目标域名固定到一个已知的服务器地址，并显式指定出口接口、禁用代理、强制 IPv4：

```
curl -4 --noproxy "*" --interface 172.31.165.86 \
    --resolve www.baidu.com:80:182.61.200.108 \
    -o baidu.html -w "%{http_code} %{size_download} %{time_total}\n" \
    http://www.baidu.com/
```

命令返回 HTTP 200，Content-Length 2381 字节，实际下载 2381 字节，全程耗时约 0.855 秒，与抓到的报文长度一致。抓包参数按手册设置：接口选实际联网的 WLAN，链路层 Ethernet，捕获过滤器 tcp port 80，关闭混杂模式，勾选网络名称解析。这一步对应手册的 Turn-in 第 1 项。

需要说明的是，原始迹线 Lab3_HTTP.pcapng 共 108 帧，其中包含其他应用程序的背景 HTTP 流量。为避免把无关连接混入分析，本次按 tcp.stream == 0 导出目标连接为 Lab3_HTTP_target.pcapng（19 帧：本机发送 10 帧、接收 9 帧）。导出过程未改写任何报文，帧号仍为 1–19，后续分析均以该文件为

准。

项目	实测配置
操作系统	Windows 11 25H2（build 26200），64 位
抓包与分析	Wireshark / TShark 4.6.8，Npcap 1.88
命令行工具	curl 8.21.0；Windows tracert；系统 ICMP API
捕获接口	WLAN：MediaTek Wi-Fi 6E MT7922 (RZ616) 160MHz Wireless LAN Card
接口地址	IPv4 172.31.165.86；网关 172.31.0.1；本机 MAC 14:ac:60:56:df:85
链路下一跳 MAC	54:c6:ff:7b:38:02
目标服务器	www.baidu.com → 182.61.200.108（用 --resolve 固定）
链路层类型	Ethernet（关闭混杂模式，启用网络名称解析）

本轮实验没有修改任何路由表、VPN 或系统网络配置；路径探测只在应用层指定源地址，属于普通用户态操作。

本节只记录实验过程与实测现象：每一步做了什么、看到什么结果、出现什么异常以及如何排查。手册要求提交的题目与答案统一放在第五节，便于对照评分。

4.1 单播 HTTP 流量的获取与定位

按手册要求选择实际联网的 WLAN 接口，链路层为 Ethernet，捕获过滤器填 tcp port 80，关闭混杂模式，并勾选网络名称解析，如图 1 与图 2 所示。过滤器的作用是把迹线限制在标准 Web 流量上，避免无关协议干扰；关闭混杂模式则只记录本机收发的帧。

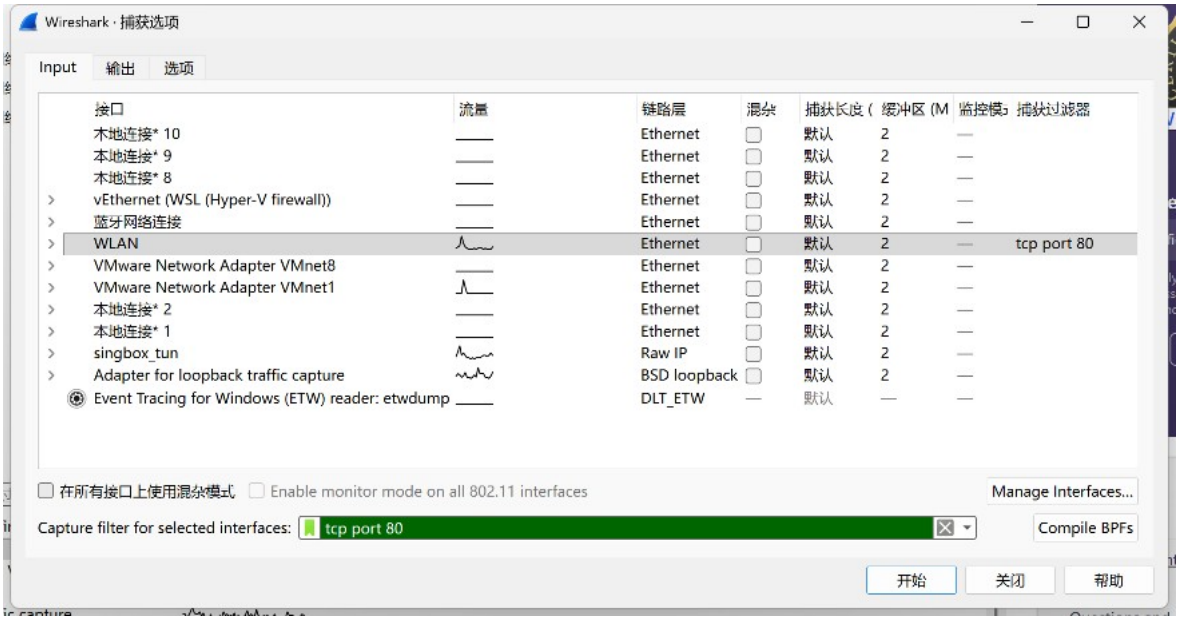


图 1 捕获选项：接口 WLAN、链路层 Ethernet、捕获过滤器 tcp port 80、混杂模式未勾选（来源：Lab3_HTTP.pcapng 抓包前的实际设置）

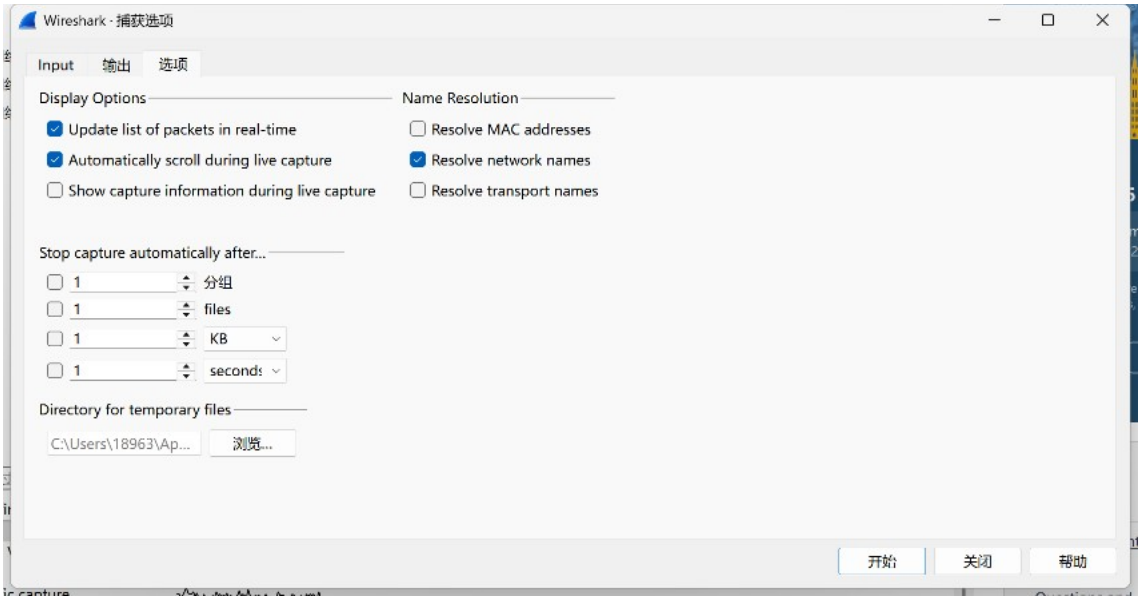


图 2　网络名称解析已启用，地址列可显示可识别的主机名（来源：同一次抓包设置）

开始捕获后重新执行 curl，命令成功后停止捕获。原始迹线共 108 帧，包含背景流量；按 tcp.stream == 0 导出目标连接后得到 19 帧，构成一次完整的 HTTP 交互：TCP 三次握手、HTTP 请求与响应、四次挥手，如表 1 所示。图 3 为导出后的目标连接列表，图 4 为跟随该 TCP 流看到的实际内容，可以看到 GET / HTTP/1.1、200 OK 与 Content-Length: 2381，与 curl 的输出互相印证。

阶段	帧号	方向	IP Total Length	说明
握手	1–3	本机 ⇄ 服务器	52 / 52 / 40	SYN、SYN/ACK、ACK
请求	4	本机 → 服务器	117	HTTP GET（含 77 B 请求）
响应	5–9, 14	服务器 → 本机	40 / 1400 / 216 / 373 / 373 / 829	ACK 与数据段，含一次重传
确认与挥手	10–13, 15–19	双向	52 / 40 / 60	Dup ACK、FIN/ACK、ACK

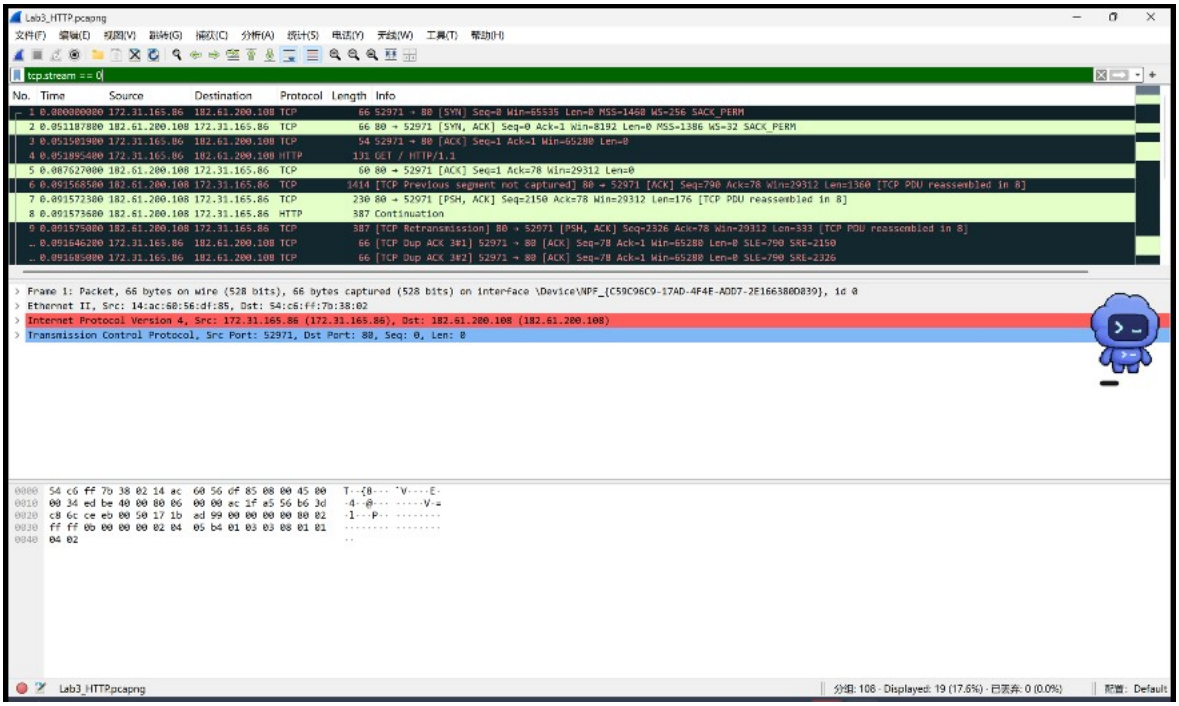


图 3　目标连接列表与显示过滤器 tcp.stream == 0：19 帧全部属于本次 HTTP 交互（来源：Lab3_HTTP_target.pcapng）



图 4　跟随 TCP 流的实际内容：GET / HTTP/1.1 与 HTTP/1.1 200 OK、Content-Length 2381（来源：Lab3_HTTP_target.pcapng 第 4 帧与第 14 帧所在流）

4.2　IPv4 首部的逐字段检查

选取本机发出的 HTTP GET 报文（第 4 帧）展开 IPv4 首部，如图 5 所示。该帧的 IP 首部为最小长度 20 字节，没有选项字段，因此可以直接与原始字节逐一对照。首部原始字节为 45 00 00 75 ed c0 40 00 80 06 00 00 ac 1f a5 56 b6 3d c8 6c，各字段对应关系如表 2。

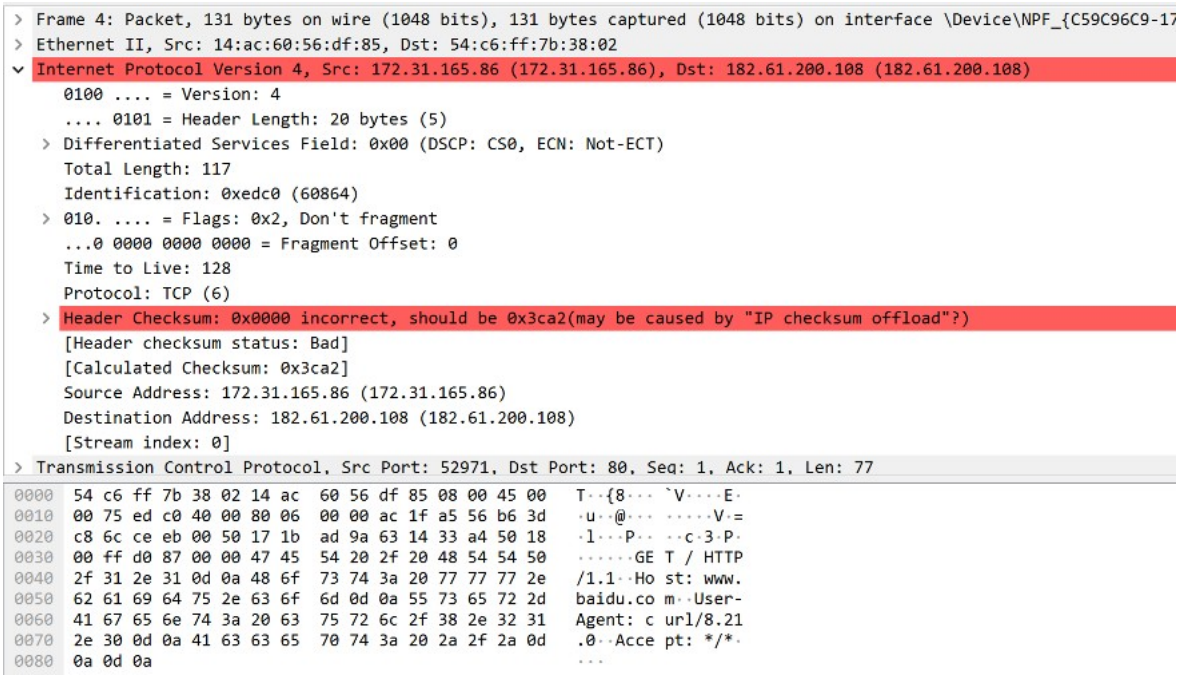


图 5　第 4 帧（HTTP GET）的 IPv4 首部各字段与对应的原始字节高亮，TTL = 128、Total Length = 117、首部校验和为 0x0000（来源：Lab3_HTTP_target.pcapng 第 4 帧）

字段	字节	长度	实测值	说明
Version + IHL	0	1 B	0x45	Version=4，IHL=5
Differentiated Services	1	1 B	0x00	DSCP CS0，ECN Not-ECT
Total Length	2–3	2 B	0x0075 = 117	首部 20 + 载荷 97
Identification	4–5	2 B	0xedc0	分片重组时用于归组
Flags + Fragment Offset	6–7	2 B	0x4000	DF=1，MF=0，Offset=0
Time To Live	8	1 B	128	本机 IPv4 默认值
Protocol	9	1 B	6	上层为 TCP
Header Checksum	10–11	2 B	0x0000	抓包点为网卡卸载，未填值
Source Address	12–15	4 B	172.31.165.86	本机 WLAN
Destination Address	16–19	4 B	182.61.200.108	远端服务器

关于首部校验和为 0 的现象需要谨慎解读：本机发出的所有 IP 报文在抓包点校验和都是 0x0000，Wireshark 因此把它们标为校验和有误。手册对此已有说明——这是校验和卸载（checksum offload），操作系统把计算与填写交给网卡在发出时完成，而抓包发生在网卡处理之前。因此不能据此断言线上首部真的错误；接收方向的报文校验和非零且被判定为正确，第 5.4 节将用它来做验算。

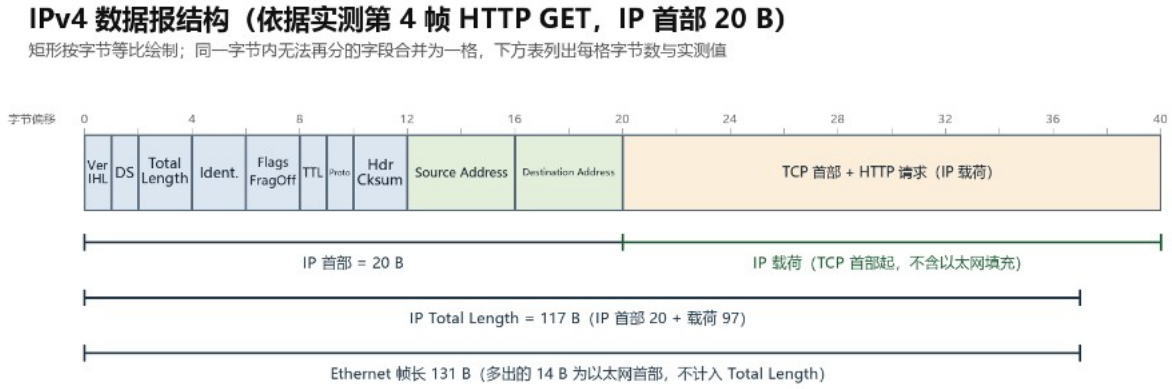
五、必做题目与解答

本节集中列出手册要求提交的题目与本次作答结果。题目原文以「题目原文」标出，答案紧接其后；涉及的图、表编号与第五节内部编号一致。

5.1 必做题目一：绘制 IP 包结构图

题目原文（Step 3）：绘出所研究 IP 包的示意图，显示 IP 首部各字段的位置与字节长度；由于无法轻易判定字节内部的位宽，把占用同一字节的字段合并表示；并标出以太网首部与 IP 载荷的范围。

依据第 4 帧的实测字段绘制 IP 数据报结构图，如图 6。图中矩形按字节等比绘制，把无法再分的字段（如 Version 与 IHL 共用第 0 字节、Flags 与 Fragment Offset 共用第 6、7 字节）合并为一格；下方表格给出每格的长度与实测取值。本图为依据实测数据绘制的示意图，不是软件截图。



字段（共用字节者合并）	字节偏移	长度	实测值（第 4 帧）
Version + IHL	0	1 B	0x45：Version=4，IHL=5
Differentiated Services	1	1 B	0x00（DSCP CS0，ECN Not-ECT）
Total Length	2–3	2 B	0x0075 = 117 B
Identification	4–5	2 B	0xedc0
Flags + Fragment Offset	6–7	2 B	0x4000：DF=1，MF=0，Offset=0
Time To Live	8	1 B	128
Protocol	9	1 B	6（TCP）
Header Checksum	10–11	2 B	0x0000（网卡校验和卸载，抓包点尚未填）
Source Address	12–15	4 B	172.31.165.86
Destination Address	16–19	4 B	182.61.200.108

说明：本图为依据实测字段绘制的示意图，不是 Wireshark 截图；“Hdr Cksum = 0x0000”属校验和卸载现象，正文有说明。

图 6 IPv4 数据报结构（依据 Lab3_HTTP_target.pcapng 第 4 帧实测，IP 首部 20 B；本图为绘制示意图）

与教材的 IPv4 首部图对照后没有发现结构差异，但有两处值得记录：一是首部长度并不是独立字段，而是与版本号共享同一个字节，只能以 4 字节为单位表示；二是校验和在抓包点为空，这属于实现细节而非协议本身的差异。

5.2 必做题目二：IPv4 六道问题

题目原文（Step 3）：结合迹线中的 IP 包回答以下六个问题。逐题作答如下。

问题 1：本机与远端服务器的 IP 地址是什么？本机 WLAN 接口地址为 172.31.165.86，远端服务器为 182.61.200.108。这两个地址可以从迹线中任一收发方向报文的 Source Address 与 Destination Address 字段直接读出，方向由地址顺序即可区分。

问题 2：Total Length 包含 IP 首部还是仅包含 IP 载荷？两者都包含，即 IP 首部长度加上 IP 载荷长度，但不包含以太网首部。以第 4 帧为例，Total Length = 117 B，而首部为 20 B、TCP 首部 20 B、HTTP 请求 77 B，20 + 20 + 77 = 117，正好吻合。该帧的以太网帧长为 131 B，131 − 117 = 14 B 恰好是以太网首部，可见链路层首部不计入 Total Length。

问题 3：Identification 字段如何变化？本机发送方向按顺序为 0xedbe、0xedbf、0xedc0、0xedc1、0xedc2、0xedc3、0xedc4、0xedc5、0xedc6、0xedc7，逐包递增 1；接收方向依次为 0xedbe（第 2 帧）、0xb73c、0xb73e、0xb73f、0xb740、0xb741、0xb744、0xb745、0xb746，存在跳号。因此可以说：同一方向、由同一台主机产生的报文其 Identification 常呈递增趋势，但两方向的取值互相独立，且接收方向出

现跳号，所以既不能说它在整个 TCP 连接中保持不变，也不能断言两方向绝无重复。需要强调的是 Identification 不是 TCP 连接号，它服务于分片重组：同一原始数据报拆成的多个分片共享该值，接收端依据源/目的地址、Protocol 与 Identification 等把它们归到一组。对未分片的 atomic datagram，RFC 6864 并不要求每个包的取值都不相同。

问题 4：本机发出的报文 TTL 初值是多少？是最大值吗？本机发送的所有报文 TTL 均为 128，不是 8 位字段的最大值 255。这说明发送方主动选择了一个较低的初值（Windows 的 IPv4 默认值）。需要与路径探测区分：在绑定源地址的 traceroute 中，探测报文被故意设置成 TTL = 1、2、3……17，那是为了实现逐跳探测，与本机普通数据流的 TTL 初值不是一回事，两者不能混为一谈。

问题 5：如何判断一个包没有被分片？关键在于 Fragmentation 相关的三个信息：MF（More Fragments）标志为 0 且 Fragment Offset 为 0。本迹线 19 个包全部满足 MF=0、Offset=0。需要注意 DF（Don't Fragment）位在本迹线中虽然都是 1，但 DF=1 并不是“未分片”的必要条件：一个报文完全可以是 DF=0 却因为长度未超过 MTU 而没有分片。反过来，若 MF=0 但 Offset>0，则说明它是某个已分片数据报的最后一片。因此判据应是 MF=0 且 Offset=0。另外 Fragment Offset 的单位是 8 字节而不是 1 字节。

问题 6：IP 首部长度是多少，在首部长度字段中如何编码？本次所有报文的首部长度都是 20 B，即不含选项的最小首部。首部的第 0 字节为 0x45，高 4 位是版本号（0100 = 4），低 4 位是 IHL（0101 = 5）。IHL 表示的是“以 4 字节为单位的首部长度”，因此实际字节数 = IHL × 4 = 5 × 4 = 20 B。正因为只有 4 位，IHL 的最大值为 15，对应首部最长 60 B，这既是选项字段长度受限的原因，也是首部长度必须以 4 字节对齐的原因。

5.3 必做题目三：Internet 路径图与组织标注

题目原文（Step 4）：依据 traceroute 输出绘制网络路径示意图，标明本机与远端服务器的 IP 地址，以及路径上路由器的跳数编号；并利用域名信息标注每一跳所属的真实组织；无法解析的跳留空。

手册用 traceroute 发现源到目的之间的路由器级路径，其原理是让探测报文以递增的 TTL 发出，第 1 台路由器把 TTL 减到 0 后回送 ICMP Time Exceeded，据此逐个暴露路径上的路由器。图 7 为迹线中 ICMP Time Exceeded 报文的实际内容：外层源地址 10.100.4.1、目的为本机，ICMP Type=11、Code=0，报文内部还引用了被丢弃的那个 Echo 请求的 IP 首部与原始数据。

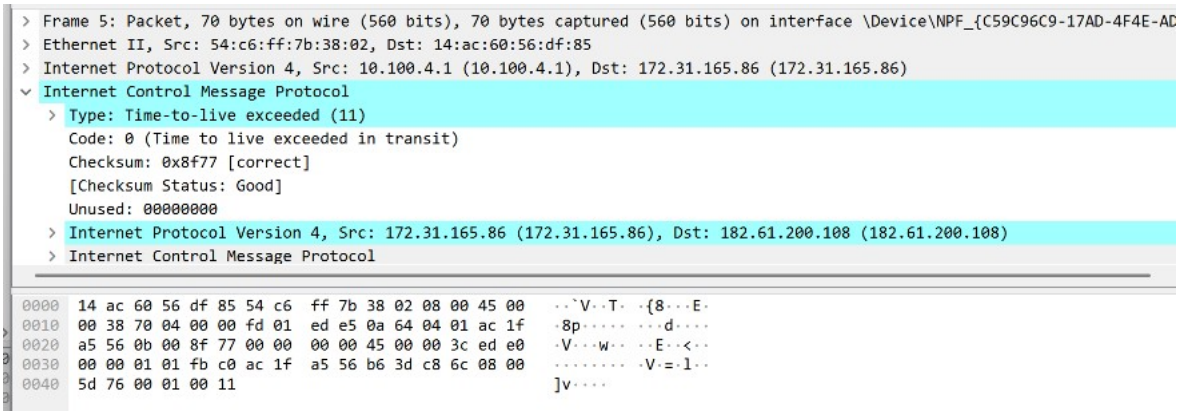


图 7 ICMP Time Exceeded（Type 11 / Code 0）：外层源地址 10.100.4.1，内部引用本机发往 182.61.200.108 的原始 Echo 请求（来源：Lab3_Path_WLAN.pcapng 第 5 帧）

本次路径测量遇到一个必须交代的异常。直接执行 Windows 自带的 tracert -4 -h 30 -w 700 182.61.200.108，得到的输出只有两跳：

```
  1    *        62 ms    *        182.61.200.108
  2    33 ms    32 ms    33 ms    182.61.200.108
```

这显然不是真实的物理路径——不可能第 1 跳就到达终点。检查发现本机启用了 singbox_tun，系统默认 IPv4 路由指向 172.18.0.2，因此该命令发出的 Echo 请求在 WLAN 接口处被捕获时 TTL 全部

是 128，并没有逐跳递增，说明探测流量被本机隧道接管后由隧道重新封装，不能反映 WLAN 上的真实路径。整个排查过程中没有修改路由表、VPN 或任何系统网络设置。

为绕开隧道，改用微软文档记载的 IcmpSendEcho2Ex 接口，在调用时把源地址显式绑定为 WLAN 的 172.31.165.86，并以 900 ms 超时、每跳 3 次的方式发送逐 TTL 的 ICMP 探测。这样探测报文直接以 WLAN 地址发出，同时在 WLAN 上抓包交叉验证。因此下面给出的结果应准确称为“绑定 WLAN 源地址的 traceroute”，而不是 Windows tracert.exe 的输出。

本次共探测 17 跳，捕获到 51 个 Echo 请求、23 个 Time Exceeded 与 2 个终点 Echo Reply，TTL 从 1 到 17 每个值恰好 3 个真实探测。图 8 为其中 TTL = 1、2、3 的三次探测及其应答，可看到过滤器已排除 ICMP 引用包中的内层 IP，避免把内层地址误当成外层源地址；图 9 为同一视图的状态栏计数。

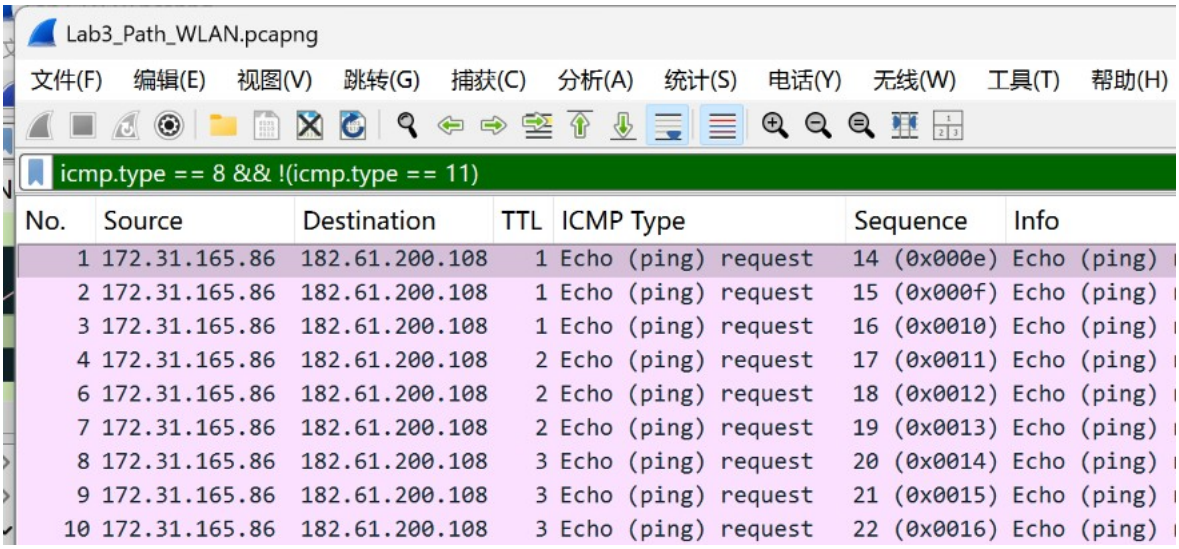


图 8　TTL = 1、2、3 的逐跳探测及其应答，过滤器排除了 ICMP 引用包的内层 IP（来源：Lab3_Path_WLAN.pcapng）

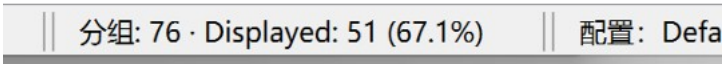


图 9　同一路径窗口底部状态栏：合计 76 帧，当前显示 51 帧（来源：Lab3_Path_WLAN.pcapng 的同一视图局部）

到 182.61.200.108 的 Internet 路径（绑定 WLAN 源地址的 ICMP 探测）

共 17 跳；每跳 3 次探测，星号表示 900 ms 内无应答，不代表该跳不存在

跳数	IP 地址	三次 RTT (ms)	组织（来源见正文说明）
1	— 无应答 —	— / — / —	未知
2	10.100.4.1	21 / — / —	私网地址，无登记信息
3	— 无应答 —	— / — / —	未知
4	202.120.95.246	5 / 4 / 6	CERNET（APNIC）
5	202.120.95.254	7 / 6 / 6	CERNET（APNIC）
6	— 无应答 —	— / — / —	未知
7	101.4.116.61	66 / 4 / 6	BJR-CERNET（APNIC）
8	101.4.118.249	6 / 6 / 5	BJR-CERNET（APNIC）
9	— 无应答 —	— / — / —	未知
10	219.224.103.65	— / 69 / 68	BJR-CERNET（APNIC）
11	101.4.129.142	32 / 30 / 30	BJR-CERNET（APNIC）
12	182.61.255.32	— / — / 33	Baidu（APNIC）
13	182.61.254.169	39 / 30 / 32	Baidu（APNIC）
14	— 无应答 —	— / — / —	未知
15	— 无应答 —	— / — / —	未知
16	— 无应答 —	— / — / —	未知
17	182.61.200.108	— / 32 / 30	Baidu 服务器（终点）

起点：本机 WLAN 172.31.165.86；终点：182.61.200.108（百度服务器）

组织名称来自 APNIC RDAP 地址登记查询，是登记归属而非物理位置或域名反解；星号跳未做推断。本图为依据 source_trace.json 绘制的示意图。

图 10　到 182.61.200.108 的 Internet 路径，共 17 跳，“无应答”为 900 ms 内未收到回复（本图为依据 source_trace.json 绘制的示意图）

关于组织标注的来源必须说明清楚：本次所有路由器的反向域名（PTR）解析均未成功，因此组织名称不是来自域名反解，而是查询 APNIC 的地址登记信息得到。具体为 RDAP 服务：202.120.95.246 属于 CERNET-CN（202.112.0.0/13 段）、101.4.116.61 属于 BJR-CERNET（101.4.0.0/14 段）、219.224.103.65 属于 BJR-CERNET（219.224.0.0/15 段）、182.61.255.32 属于 Baidu（182.61.0.0/16 段）。这类标签说明的是地址的登记归属，既不等于设备的物理位置，也不是域名反解的结果。第 2 跳 10.100.4.1 属于私网地址，没有公开登记信息，只能标注为未知。

路径图中的星号只表示在规定的 900 ms 内没有收到对应回复。它不能证明该跳不存在，也不能仅凭它断言那台路由器关闭了 ICMP；常见原因还包括路由器限速 ICMP、回复走了不同路径或优先级较低而被丢弃。另外，ICMP 探测出来的路径不保证与 TCP 数据实际走的路径完全相同，反向路径也可能不同，所以这张图描述的是本次测量条件下的转发路径。

路径整体呈现清晰的层次：出口经 10.100.4.1 进入校园网，第 4–5 跳为上海交通大学相关的 CERNET 节点，第 7–11 跳进入北京 CERNET 骨干，第 12–13 跳进入百度网络，第 17 跳到达目标服务器。这与“先出校园网、再上教育网骨干、最后进入目标运营商”的一般规律一致。

5.4　必做题目四：IP 首部校验和验算

按手册要求选取接收方向、首部 20 字节且校验和非零的报文。本次选用第 5 帧，它是服务器对本机 HTTP 请求的确认，源地址 182.61.200.108、目的地址 172.31.165.86，Wireshark 给出的首部校验和为

0xcb73 并标记 [correct]，如图 11 所示。该帧首部原始字节为：

```
45 00 00 28 b7 3c 40 00 28 06 cb 73 b6 3d c8 6c ac 1f a5 56
```

把首部按 16 位切成 10 个字，用普通加法累计，并在每一步标注对应的 IPv4 字段，结果如表 3。

序号	16 位字	对应 IPv4 字段	累计和
1	0x4500	Version、IHL、DS	0x4500
2	0x0028	Total Length	0x4528
3	0xb73c	Identification	0xfc64
4	0x4000	Flags、Fragment Offset	0x13c64
5	0x2806	TTL、Protocol	0x1646a
6	0xcb73	首部校验和（实收值）	0x22fdd
7	0xb63d	源地址前半	0x2e61a
8	0xc86c	源地址后半	0x3ae86
9	0xac1f	目的地址前半	0x45aa5
10	0xa556	目的地址后半	0x4fffb

普通累加的结果为 0x4fffb，已经超过 16 位。按 1 的补码运算规则，把最高位溢出的部分回卷加回低 16 位：0x4fffb → 高 16 位 0x0004 加上低 16 位 0xfffb，得到 0xffff。这正是 1 的补码形式的“负零”，说明校验和自洽。

为了进一步确认，再把校验和字段本身置零后重新计算一遍：其余 9 个字的和为 0x43488，回卷后为 0x348c，按位取反得到 0xcb73，与报文里实际携带的 0xcb73 以及 Wireshark 的 [correct] 判定完全一致。两次计算分别从“包含校验和应收敛到 0xffff”和“置零后取反应等于校验和”两个方向验证了同一结果。

上述数值不是从截图上抄录的，而是 analyze.py 直接读取 pcapng 的原始字节，按同样的 1 的补码规则重新计算并断言得到的，因此与界面显示互为独立验证。

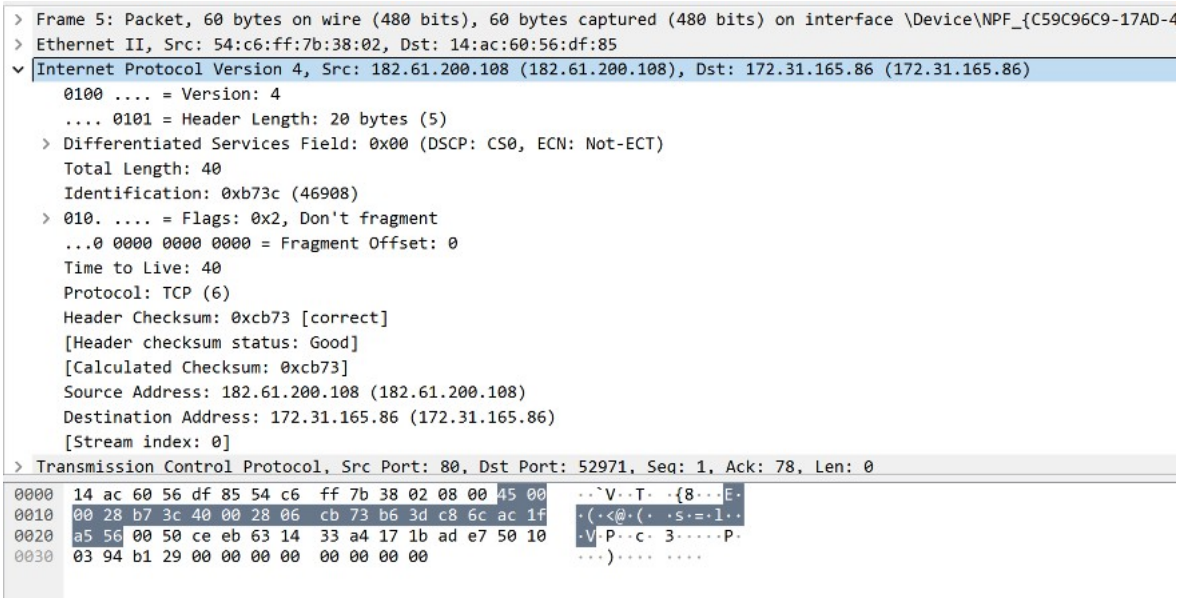
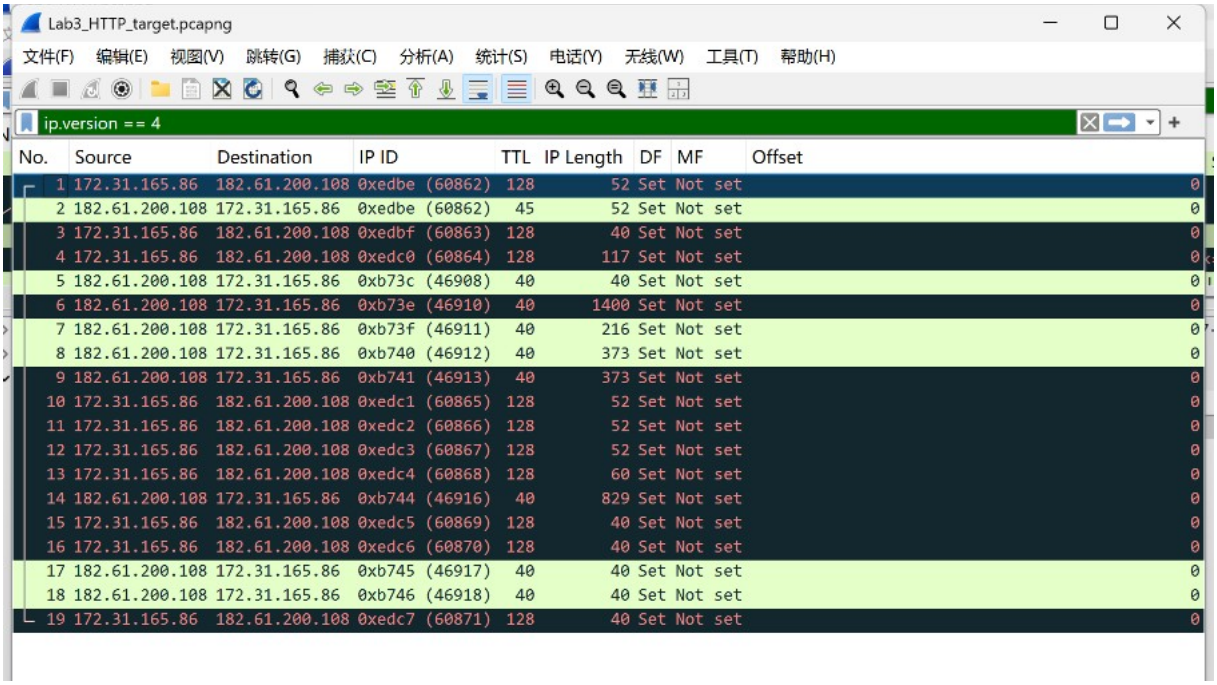


图 11　第 5 帧接收方向 IPv4 首部的 20 B 高亮，校验和 0xcb73 标记为 [correct]（来源：Lab3_HTTP_target.pcapng 第 5 帧）



No.	Source	Destination	IP ID	TTL	IP Length	DF	MF	Offset
1	172.31.165.86	182.61.200.108	0xedbe (60862)	128	52	Set	Not set	0
2	182.61.200.108	172.31.165.86	0xedbe (60862)	45	52	Set	Not set	0
3	172.31.165.86	182.61.200.108	0xedbf (60863)	128	40	Set	Not set	0
4	172.31.165.86	182.61.200.108	0xedc0 (60864)	128	117	Set	Not set	0
5	182.61.200.108	172.31.165.86	0xb73c (46908)	40	40	Set	Not set	0
6	182.61.200.108	172.31.165.86	0xb73e (46910)	40	1400	Set	Not set	0
7	182.61.200.108	172.31.165.86	0xb73f (46911)	40	216	Set	Not set	0
8	182.61.200.108	172.31.165.86	0xb740 (46912)	40	373	Set	Not set	0
9	182.61.200.108	172.31.165.86	0xb741 (46913)	40	373	Set	Not set	0
10	172.31.165.86	182.61.200.108	0xedc1 (60865)	128	52	Set	Not set	0
11	172.31.165.86	182.61.200.108	0xedc2 (60866)	128	52	Set	Not set	0
12	172.31.165.86	182.61.200.108	0xedc3 (60867)	128	52	Set	Not set	0
13	172.31.165.86	182.61.200.108	0xedc4 (60868)	128	60	Set	Not set	0
14	182.61.200.108	172.31.165.86	0xb744 (46916)	40	829	Set	Not set	0
15	172.31.165.86	182.61.200.108	0xedc5 (60869)	128	40	Set	Not set	0
16	172.31.165.86	182.61.200.108	0xedc6 (60870)	128	40	Set	Not set	0
17	182.61.200.108	172.31.165.86	0xb745 (46917)	40	40	Set	Not set	0
18	182.61.200.108	172.31.165.86	0xb746 (46918)	40	40	Set	Not set	0
19	172.31.165.86	182.61.200.108	0xedc7 (60871)	128	40	Set	Not set	0

图 12　19 帧的源/目的地址、Identification、TTL、IP Length、DF、MF、Offset 全列表，用于核对分片标志与标识变化（来源：Lab3_HTTP_target.pcapng）

手册要求提交的题目均已单独作答，位置如下，便于对照评分：

必做题目索引

手册要求	作答位置
1. 成功获取同一远端服务器的 HTTP 网页，捕获 tcp port 80 流量；记录 traceroute	实测过程见 4.1 节；curl 取得 HTTP 200，Content-Length 2381；路径结果见 5.3 节图 10
2. 检查 IPv4 首部全部字段及字节	实测过程与原始字节见 4.2 节，字段逐项对应见该节表 2
3. 绘制 IP 包结构图，回答六题	结构图见 5.1 节图 6；六题答案见 5.2 节
4. 绘制按跳编号的路径图并标注组织	见图 10（5.3 节），共 17 跳；组织标注来自 APNIC 登记信息
5. 选取接收方向、非零校验和、20 B 首部的包，把 10 个 16 位字对应到字段并验证 0xffff	见 5.4 节表 3：回卷后得 0xffff；置零复算得 0xcb73，双向一致

六、实验结果总结

6.1　实验结论

关于 IPv4 首部：本次所有报文首部长度均为 20 B，无选项字段；首字节 0x45 同时编码版本号与首部长度，IHL 以 4 字节为单位，最大值 15 对应 60 B。Total Length 统计 IP 首部与载荷之和，不含链路层首部。

关于标识与分片：本机方向 Identification 逐包递增，接收方向存在跳号，说明它只是分片重组时用于归组同一数据报的标识，并非连接号；19 个包全部满足 MF=0 且 Offset=0，因此都未分片，其中 DF=1 是发送方的策略选择而不是未分片的判据。

关于 TTL：本机普通数据流的 TTL 初值为 128，低于 8 位字段上限 255；接收方向报文 TTL 为

40，首个响应为 45。这里只报告观测值——由 64 减 TTL 直接推算跳数是不严谨的，因为无法确认对端的初始 TTL 一定是 64，而且双向路由可能不同。

关于校验和：接收方向首部校验和经两次独立计算确认正确；本机方向校验和为 0 是校验和卸载造成的观测现象，不代表线上首部有误。

关于与教材的差异：结构上没有发现差异，但有一处实现层面的现象值得记住——协议规定的字段未必在抓包点就已填好，校验和卸载就是典型例子。这提醒我们在用抓包结论判断“对错”之前，要先确认抓包位置相对于协议栈处理的位置。

6.2 实验体会

这次实验最主要的收获是意识到“看到的”和“实际发出的”可能不是一回事。本机 IP 校验和全为 0，如果只看抓包就下结论，会误判为协议实现错误；同样，直接运行 tracert 得到“两跳到达”，如果照抄就成了错误结论，实际原因是隧道接管了探测流量。两处都说明：异常现象要先定位到观测位置和处理时序，再谈结论。

另一处体会是描述数据时要留有余地。Identification 在本机方向确实逐包递增，但接收方向有跳号，因此报告里只写“呈递增趋势”而不断言每包加一；星号跳也只写“规定时间内无应答”，不推断该路由器关闭了 ICMP。把结论限制在证据支持的范围内，比给出一个漂亮但过度的断言更可靠。

最后是独立复核的价值。校验和既可以让 Wireshark 判定，也可以自己按 1 的补码算一遍；两条路径得到同一个结果，才算真正验证。本报告的字段与统计量均由脚本从 pcapng 原始字节重新解析，与界面显示互为交叉验证，而不是从截图抄写。

七、附录

附录 A 迹线与日志文件

文件	说明
Lab3_HTTP_target.pcapng	目标 HTTP 连接，19 帧，报告主要依据
Lab3_HTTP.pcapng	抓包原始迹线 108 帧（含背景流量），仅作留存
Lab3_Path_WLAN.pcapng	绑定 WLAN 源地址的路径探测，76 帧
Lab3_Traceroute.pcapng	普通 tracert 的 WLAN 迹线，隧道干扰证据
logs/Lab3_HTTP_target.tsv	逐帧字段提取结果
logs/Lab3_Path_WLAN.tsv	路径探测字段提取（-E occurrence=f）
logs/source_trace.json	17 跳路径结构，图 10 的数据来源
logs/rdap_*.json	APNIC 地址登记查询原始结果
facts.json / analyze.py	字段提取与校验和独立验算脚本及结果

附录 B 关键命令

```
# 抓取目标网页（固定服务器地址、指定接口、禁用代理、强制 IPv4）
curl -4 --noproxy "*" --interface 172.31.165.86 \
     --resolve www.baidu.com:80:182.61.200.108 -o baidu.html http://www.baidu.com/


# 导出目标 TCP 连接
tshark -r Lab3_HTTP.pcapng -Y "tcp.stream == 0" -w Lab3_HTTP_target.pcapng


# 提取 IPv4 字段用于核对（避免把 ICMP 引用包的内层 IP 当成外层）
```

```
tshark -r Lab3_HTTP_target.pcapng -T fields -E occurrence=f \
      -e frame.number -e ip.src -e ip.dst -e ip.len -e ip.id -e ip.ttl \
      -e ip.flags.df -e ip.flags.mf -e ip.frag_offset -e ip.checksum
```

附录 C 参考资料

[1] RFC 791: Internet Protocol. https://www.rfc-editor.org/rfc/rfc791.html（IPv4 首部格式与首部校验和算法）

[2] RFC 6864: Updated Specification for the IPv4 ID Field. https://www.rfc-editor.org/rfc/rfc6864.html（atomic datagram 的 Identification 不要求逐包唯一）

[3] RFC 792: Internet Control Message Protocol. https://www.rfc-editor.org/rfc/rfc792.html（Time Exceeded 报文格式）

[4] Microsoft Learn: IcmpSendEcho2Ex function. https://learn.microsoft.com/windows/win32/api/icmpapi/nf-icmpapi-icmpsendecho2ex（绑定源地址的 ICMP 探测接口）

[5] Wireshark Foundation: IPv4 display filter reference. https://www.wireshark.org/docs/dfref/i/ip.html